INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
CAMPUS BOA VIAGEM — TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: PROGRAMAÇÃO WEB I | PROF. DR. RENATO WILLIAM RODRIGUES DE SOUZA

FitBoa Viagem — Sistema Academia

Relatório Técnico de Desenvolvimento de Sistema Web Orientado a Objetos em PHP

Tema 07: Academia (Plano: mensal, trimestral, anual | Aluno tem treinos)

Aluno:	Ivamilton
E-mail:	ivamiltonjunior@gmail.com
Repositório GitHub:	https://github.com/Ivamilton/pweb1-trabalho-oo
Linguagem e Ambiente:	PHP 8.2 / 8.3 CLI (Built-in Server) — Windows
Data da Avaliação:	04 de Novembro de 2026 (Apresentação às 13h00)

Resumo Executivo: Este documento constitui o relatório final do trabalho prático individual da disciplina de Programação Web I. Apresenta a arquitetura de software, diagramação de classes em Mermaid, análise dos conceitos fundamentais de Orientação a Objetos (OO1 a OO8), rotas da camada web (W1 a W5), implementação do bônus de persistência em arquivo JSON, validação de 13 cenários de teste automatizados e o registro crítico do uso de Modelos de Linguagem de Grande Porte (LLMs). O sistema modela a gestão completa de uma academia com planos contratuais polimórficos e rotinas semanais de treino.

1. Introdução

1.1 Contexto e Motivação

A informatização de serviços essenciais é uma demanda crescente em organizações de todos os portes. No setor de saúde e bem-estar, academias de musculação e ginástica lidam diariamente com o fluxo constante de matrículas, gestão de mensalidades e organização pedagógica das rotinas de treino. Na prática de pequenos estabelecimentos, o controle dessas entidades comumente se dá por meio de fichas físicas de papel ou anotações manuais. Esse método arcaico acarreta perdas substanciais de dados, dificuldades na conferência de inadimplência e conflitos de horários de instrutores.

O sistema **FitBoa Viagem** foi concebido para mitigar essas vulnerabilidades mediante a aplicação prática dos conceitos centrais ministrados no componente curricular Programação Web I: o funcionamento do protocolo HTTP e cabeçalhos de resposta (Aula 01), o processamento da requisição e renderização de respostas dinâmicas em PHP (Aula 02) e o paradigma da Orientação a Objetos (Aula 03).

1.2 Para Quem o Sistema Serve

A plataforma atende de forma integrada a dois agentes operacionais do ecossistema fitness:

- **Gestão Administrativa:** Permite matricular alunos associando-os a planos contratuais diferenciados (mensal, trimestral ou anual), assegurando que não haja duplicidade de cadastros por CPF e fornecendo indicadores automáticos em tempo real do faturamento mensal estimado com base nas assinaturas ativas.
- **Corpo de Instrutores e Alunos:** Permite prescrever e estruturar as sessões de treinamento semanal. O sistema impõe salvaguardas fisiológicas e organizacionais: limita a 6 o número máximo de dias de treino na semana, impede o agendamento de múltiplos treinos no mesmo dia e calcula com precisão a carga horária semanal acumulada.

O sistema oferece tanto uma interface gráfica com formulários protegidos contra injeções quanto uma rota de dados no formato padronizado JSON, viabilizando integrações futuras com catracas eletrônicas ou aplicativos móveis.

2. Requisitos do Sistema e Regras de Negócio

2.1 Funcionalidades da Aplicação Web

O sistema organiza sua navegação através do padrão de query string (?pagina=...), compartilhando um cabeçalho e menu de navegação responsivo em todas as telas: Dashboard inicial com métricas consolidadas, catálogo comparativo de planos com demonstração de polimorfismo, formulários de adesão e de agendamento de treinos com tratamento de exceções, prontuário detalhado do aluno e endpoint JSON estruturado.

2.2 Regras de Negócio e Tabela de Exceções

Para impedir estados inconsistentes, foram definidas 11 regras de domínio com disparo de Exception tratadas com try/catch:

Cód	Regra de Negócio de Domínio	Classe e Método	Mensagem Disparada na Exceção
RN01	Nome do aluno não pode ser vazio ou espaço	`Aluno::__construct`	O nome do aluno não pode ser vazio.
RN02	CPF deve possuir estritamente 11 dígitos	`Aluno::__construct`	CPF inválido. Use o formato 000.000.000-00 (11 dígitos).
RN03	Preço-base do plano deve ser estritamente > 0	`Plano::__construct`	O preço-base deve ser maior que zero.
RN04	Duração do plano deve ser de no mínimo 1 mês	`Plano::__construct`	A duração deve ser de pelo menos 1 mês.
RN05	Desconto deve pertencer ao intervalo [0.0, 1.0]	`PlanoTrimestral / Anual`	O desconto deve estar entre 0 e 1.
RN06	Nome do treino não pode ser vazio	`Treino::__construct`	O nome do treino não pode ser vazio.
RN07	Dia da semana deve ser válido (segunda a domingo)	`Treino::__construct`	Dia da semana inválido: {dia}.
RN08	Duração da sessão deve estar entre 1 e 240 min	`Treino::__construct`	A duração do treino deve ser entre 1 e 240 minutos.
RN09	Limite fisiológico máximo de 6 treinos por aluno	`Aluno::adicionarTreino`	Um aluno pode ter no máximo 6 treinos.

RN10	Vedado agendar mais de um treino no mesmo dia	`Aluno::adicionarTreino`	Já existe um treino na {dia} para este aluno.
RN11	Impedir matrícula duplicada de CPF na academia	`Academia::adicionarAluno`	Já existe um aluno cadastrado com este CPF.

3. Modelagem Orientada a Objetos

3.1 Diagrama Estrutural de Classes (Mermaid)

O diagrama a seguir sintetiza a arquitetura do domínio concebida para o tema 07 (Academia), evidenciando a hierarquia de herança e os relacionamentos de composição:

```
classDiagram
    direction TB
    Plano <|-- PlanoMensal
    Plano <|-- PlanoTrimestral
    Plano <|-- PlanoAnual
    Aluno "1" *-- "*" Treino : tem treinos
    Aluno --> Plano : possui
    Academia "1" *-- "*" Aluno : matricula

    class Plano {
        <<abstract>>
        #string nome
        #float precoBase
        #int duracaoMeses
        +__construct(string nome, float precoBase, int duracaoMeses)
        +getNome() string
        +getPrecoBase() float
        +getDuracaoMeses() int
        +calcularValorTotal()* float
        +getTipo()* string
        +paraArray() array
    }
    class PlanoMensal {
        +__construct(string nome, float precoBase)
        +calcularValorTotal() float
        +getTipo() string
    }
    class PlanoTrimestral {
        -float desconto
        +__construct(string nome, float precoBase, float desconto)
        +calcularValorTotal() float
        +getTipo() string
    }
    class PlanoAnual {
        -float desconto
        +__construct(string nome, float precoBase, float desconto)
        +calcularValorTotal() float
        +getTipo() string
    }
    class Treino {
        -string nome
        -string diaSemana
        -int duracaoMinutos
        +__construct(string nome, string diaSemana, int duracaoMinutos)
        +paraArray() array
    }
    class Aluno {
        -string nome
        -string cpf
        -Plano plano
        -array treinos
        +adicionarTreino(Treino treino) void
        +calcularCargaHorariaSemanal() int
        +paraArray() array
    }
    class Academia {
        -string nome
        -array alunos
        +adicionarAluno(Aluno aluno) void
        +calcularReceitaMensal() float
        +paraArray() array
    }
```

3.2 Justificativa da Herança e da Composição

Herança: A classe abstrata Plano define o molde comum com atributos protegidos ($nome, $precoBase e $duracaoMeses). As especializações PlanoMensal, PlanoTrimestral e PlanoAnual herdam via extends e invocam parent::__construct(...), garantindo a validação de parâmetros na origem. A relação *'é um'* é plena: um Plano Trimestral *é um* Plano que possui duração de 3 meses e política própria de precificação. A sobrescrita de calcularValorTotal() aplica dinamicamente os abatimentos comerciais (0%, 10% e 20%).

Composição: Optou-se pela composição (relação *'tem um'*) em duas frentes fundamentais: (1) Aluno contém uma lista de Treino, pois um aluno não é um treino, mas sim mantém e agenda sessões de exercício; (2) Academia contém uma lista de Aluno, agregando a carteira de sócios matriculados para fins de auditoria e cálculo da receita mensal agregada.

Encapsulamento: Nenhum atributo é público. Protected na classe mãe para acesso nas filhas e private nas demais, com leitura por getters.

4. Implementação Orientada a Objetos

Apresenta-se a concretização de cada um dos oito requisitos de Orientação a Objetos estabelecidos no edital:

OO1. Classes com Construtor, Atributos Tipados e declare(strict_types=1)

```
// Exemplo em src/Treino.php
declare(strict_types=1);

class Treino {
    private string $nome;
    private string $diaSemana;
    private int    $duracaoMinutos;

    public function __construct(string $nome, string $diaSemana, int $duracaoMinutos) {
        if (trim($nome) === '') throw new Exception('O nome do treino não pode ser vazio.');
        $dias = ['segunda', 'terca', 'quarta', 'quinta', 'sexta', 'sabado', 'domingo'];
        if (!in_array(strtolower(trim($diaSemana)), $dias, true)) {
            throw new Exception("Dia da semana inválido: {$diaSemana}.");
        }
        $this->nome = trim($nome);
        $this->diaSemana = strtolower(trim($diaSemana));
        $this->duracaoMinutos = $duracaoMinutos;
    }
}
```

OO2. Hierarquia com extends, parent::__construct e Sobrescrita de Método

```
// Exemplo em src/PlanoTrimestral.php
class PlanoTrimestral extends Plano {
    private float $desconto;

    public function __construct(string $nome, float $precoBase, float $desconto = 0.10) {
        if ($desconto < 0 || $desconto > 1) throw new Exception('O desconto deve estar entre 0 e 1.');
        parent::__construct($nome, $precoBase, 3);
        $this->desconto = $desconto;
    }

    public function calcularValorTotal(): float {
        return round($this->precoBase * $this->duracaoMeses * (1 - $this->desconto), 2);
    }
    public function getTipo(): string { return 'Trimestral'; }
}
```

OO3. Polimorfismo: foreach que Percorre Classes Diferentes

```
// Trecho de public/index.php e teste.php
$planos = [$planoMensal, $planoTrimestral, $planoAnual];
foreach ($planos as $plano) {
    // Invocação uniforme com resolução polimórfica em tempo de execução:
    $valorTotal = $plano->calcularValorTotal();
    $tipo = $plano->getTipo();
}
```

OO4. Encapsulamento com Atributos Privados/Protegidos e Getters

```
// Em src/Aluno.php: CPF é privado e imutável após construção
public function getCpfFormatado(): string {
    $c = $this->cpf;
    return substr($c, 0, 3) . '.' . substr($c, 3, 3) . '.' . substr($c, 6, 3) . '-' . substr($c, 9, 2);
}
```

4. Implementação Orientada a Objetos (Continuação)

OO5. Regras de Negócio com throw new Exception e Tratamento try/catch

```
// Disparo em src/Academia.php e captura em public/index.php:
public function adicionarAluno(Aluno $aluno): void {
    foreach ($this->alunos as $a) {
        if ($a->getCpf() === $aluno->getCpf()) throw new Exception('Já existe um aluno com este CPF.');
    }
    $this->alunos[] = $aluno;
}

try {
    $academia->adicionarAluno($aluno);
    $mensagemSucesso = "Aluno cadastrado com sucesso!";
} catch (Exception $e) {
    $mensagemErro = $e->getMessage();
}
```

OO6. Composição com Inclusão e Cálculo sobre Lista

```
// Em src/Aluno.php:
public function adicionarTreino(Treino $treino): void {
    if (count($this->treinos) >= 6) throw new Exception('Um aluno pode ter no máximo 6 treinos.');
    foreach ($this->treinos as $t) {
        if ($t->getDiaSemana() === $treino->getDiaSemana()) {
            throw new Exception("Já existe um treino na {$treino->getDiaSemana()} para este aluno.");
        }
    }
    $this->treinos[] = $treino;
}

public function calcularCargaHorariaSemanal(): int {
    $total = 0;
    foreach ($this->treinos as $treino) { $total += $treino->getDuracaoMinutos(); }
    return $total;
}
```

OO7. Método paraArray() em Todas as Classes Envolvidas no JSON

```
// Em src/Aluno.php:
public function paraArray(): array {
    $treinosArray = [];
    foreach ($this->treinos as $treino) { $treinosArray[] = $treino->paraArray(); }
    return [
        'nome' => $this->nome, 'cpf' => $this->getCpfFormatado(),
        'plano' => $this->plano->paraArray(), 'treinos' => $treinosArray,
        'cargaHorariaSemanal' => $this->calcularCargaHorariaSemanal()
    ];
}
```

OO8. Isolamento de Arquivos em src/ e Carregamento com require_once

```
// Carregamento modular canônico no topo de public/index.php e teste.php:
require_once __DIR__ . '/../src/Plano.php';
require_once __DIR__ . '/../src/PlanoMensal.php';
require_once __DIR__ . '/../src/PlanoTrimestral.php';
require_once __DIR__ . '/../src/PlanoAnual.php';
require_once __DIR__ . '/../src/Treino.php';
require_once __DIR__ . '/../src/Aluno.php';
require_once __DIR__ . '/../src/Academia.php';
```

5. Camada Web e Evidências com curl.exe

A camada web segue estritamente as diretrizes W1 a W5: roteamento por query string com cabeçalho/menu unificados e status 404; rota JSON com Content-Type application/json; formulários POST protegidos por try/catch; proteção contra XSS via htmlspecialchars(); e lógica no topo do script.

Bônus de Persistência em JSON: Os dados criados via formulário são automaticamente persistidos no arquivo data/academia.json através de file_put_contents e recuperados via json_decode, sobrevivendo à reinicialização do servidor.

5.1 Rota Válida: Tabela de Planos (GET /?pagina=planos — 200 OK)

```
$ curl.exe -i "http://127.0.0.1:8000/?pagina=planos"
HTTP/1.1 200 OK
Host: 127.0.0.1:8000
Content-type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="pt-BR">
...
  <main>
    <h2>Planos Disponíveis</h2>
    <table>
      <thead><tr><th>Tipo</th><th>Nome</th><th>Duração</th><th>Valor Total</th><th>Valor/Mês</th></tr></thead>
      <tbody>
        <tr><td>Mensal</td><td>Plano Mensal</td><td>1 mês</td><td>R$ 89,90</td><td>R$ 89,90</td></tr>
        <tr><td>Trimestral</td><td>Plano Trimestral</td><td>3 meses</td><td>R$ 242,73</td><td>R$ 80,91</td></tr>
        <tr><td>Anual</td><td>Plano Anual</td><td>12 meses</td><td>R$ 863,04</td><td>R$ 71,92</td></tr>
      </tbody>
    </table>
  </main>
```

5.2 Rota Inexistente: Erro 404 (GET /?pagina=inexistente — 404 Not Found)

```
$ curl.exe -i "http://127.0.0.1:8000/?pagina=inexistente"
HTTP/1.1 404 Not Found
Content-type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="pt-BR">
...
  <main>
    <h2>404 — Página não encontrada</h2>
    <p>A página que você procura não existe. <a href="?pagina=inicio">Voltar ao início</a>.</p>
  </main>
```

5.3 Rota de Integração JSON (GET /?formato=json — application/json)

```
$ curl.exe -i "http://127.0.0.1:8000/?formato=json"
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
    "nome": "FitBoa Viagem",
    "totalAlunos": 1,
    "receitaMensal": 89.9,
    "alunos": [
        {
            "nome": "Carlos Silva",
            "cpf": "123.456.789-01",
            "plano": { "tipo": "Mensal", "nome": "Plano Mensal", "precoBase": 89.9, "valorTotal": 89.9 },
            "treinos": [], "cargaHorariaSemanal": 0
        }
    ]
}
```

5.4 Formulário POST com Dado Inválido e Mensagem de Erro na Página

```
$ curl.exe -i -X POST -d "acao=cadastrar_aluno&nome=&cpf=12345678901&plano=mensal" \
  "http://127.0.0.1:8000/?pagina=cadastrar"
HTTP/1.1 200 OK
Content-type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="pt-BR">
...
  <main>
    <div class="mensagem erro">O nome do aluno não pode ser vazio.</div>
    <h2>Cadastrar Aluno</h2>
```

6. Roteiro de Testes Automatizados no Terminal

O arquivo teste.php foi elaborado como uma suíte de asserções executável via linha de comando (php teste.php), comprovando cada requisito de domínio de forma isolada:

```
=== SISTEMA ACADEMIA — ROTEIRO DE TESTES ===

=== Teste 1: Criação dos planos e polimorfismo ===
  Mensal (Plano Mensal): R$ 89,90 por 1 mês(es)
  Trimestral (Plano Trimestral): R$ 242,73 por 3 mês(es)
  Anual (Plano Anual): R$ 863,04 por 12 mês(es)

=== Teste 2: Criação de aluno com dados válidos ===
  Aluno criado: Maria da Silva, CPF: 123.456.789-01 | Plano: Mensal

=== Teste 3: Recusa — nome do aluno vazio ===
  OK, recusado: O nome do aluno não pode ser vazio.

=== Teste 4: Recusa — CPF inválido ===
  OK, recusado: CPF inválido. Use o formato 000.000.000-00 (11 dígitos).

=== Teste 5: Recusa — preço-base negativo ===
  OK, recusado: O preço-base deve ser maior que zero.

=== Teste 6: Composição — adicionando treinos ===
  Treinos de Maria da Silva:
    - Treino A - Peito e Tríceps (segunda, 60 min)
    - Treino B - Costas e Bíceps (quarta, 75 min)
    - Treino C - Pernas (sexta, 90 min)
  Carga horária semanal: 225 minutos

=== Teste 7: Recusa — treino duplicado no mesmo dia ===
  OK, recusado: Já existe um treino na segunda para este aluno.

=== Teste 8: Recusa — dia da semana inválido ===
  OK, recusado: Dia da semana inválido: feriado. Use: segunda, terca, quarta, quinta, sexta, sabado, domingo.

=== Teste 9: Recusa — duração do treino acima de 240 min ===
  OK, recusado: A duração do treino deve ser entre 1 e 240 minutos.

=== Teste 10: Academia — alunos e receita mensal ===
  Total de alunos: 3 | Receita mensal estimada: R$ 242,73

=== Teste 11: Recusa — CPF duplicado na academia ===
  OK, recusado: Já existe um aluno cadastrado com este CPF.

=== Teste 12: paraArray() — saída JSON ===
  [Estrutura JSON válida gerada com sucesso]

=== Teste 13: Recusa — máximo de 6 treinos por aluno ===
  OK, recusado: Um aluno pode ter no máximo 6 treinos.

=== TODOS OS TESTES CONCLUÍDOS ===
```

Análise Técnica das Asserções:

- **Asserção 1 (Polimorfismo):** Demonstra que instâncias de PlanoMensal, PlanoTrimestral e PlanoAnual respondem à mesma invocação calculando valores específicos com base em seus abatimentos.
- **Asserções 3, 4 e 5 (Sanitização de Matrícula):** Comprovam que nomes vazios, CPFs incompletos e taxas financeiras negativas são imediatamente bloqueados no construtor.
- **Asserção 6 (Composição e Agregação):** Valida a adição de 3 treinos independentes e a computação precisa da carga horária semanal (60 + 75 + 90 = 225 minutos).
- **Asserções 7, 8, 9 e 13 (Integridade da Rotina de Treino):** Asseguram que um aluno não possa realizar dois treinos no mesmo dia, não insira dias inexistentes, não exceda 240 minutos por treino e não ultrapasse 6 sessões de treino na semana.
- **Asserções 10 e 11 (Controle Corporativo):** Aferem o cálculo da receita contábil mensal e a proteção contra cadastros redundantes de um mesmo CPF na academia.

7. Ferramentas Utilizadas

A confecção do sistema e de seus artefatos empregou ferramentas alinhadas com as práticas vigentes na indústria de desenvolvimento de software:

Ferramenta	Versão / Ecossistema	Papel Desempenhado no Projeto
PHP CLI	8.2.33 / 8.3 ZTS (Windows)	Motor de execução da linguagem. Utilizado para executar o roteiro de testes (teste.php), verificar erros de tipagem sob declare(strict_types=1) e executar o servidor web embutido (php -S localhost:8000 -t public).
VS Code	1.95+ (Microsoft)	Ambiente de desenvolvimento integrado (IDE) principal, utilizado para edição modular do código, destaque de sintaxe, inspeção do sistema de arquivos e depuração.
Git & GitHub	2.40+ / Repositório Remoto	Controle de versionamento com histórico atômico segundo a convenção Conventional Commits (feat, test, style, chore), ramificações de funcionalidades e marcação de tags nos marcos cronológicos (v0.1, v0.2, v1.0).
curl.exe	8.0+ Nativo Windows	Ferramenta de linha de comando para validação HTTP em baixo nível: verificação dos cabeçalhos de resposta, código 200 OK, código 404 Not Found, cabeçalho de Content-Type do JSON e testes de POST.
Navegador & DevTools	Google Chrome / Edge	Interface para teste de experiência do usuário, conferência do layout CSS responsivo, submissão interativa de formulários e análise da aba Network.
Modelos LLM	Claude Opus 4.6 / Gemini 3.8	Ferramentas de Inteligência Artificial utilizadas de forma pedagógica para revisão de modelagem de domínio, auxílio em sintaxe Mermaid, sugestões de casos de teste e geração deste relatório.
Gravador de Tela	OBS Studio / Xbox Game Bar	Captação de áudio e vídeo de alta fidelidade para produção do vídeo explicativo de até 10 minutos demonstrando a execução do sistema e inspeção detalhada do código-fonte.

8. Uso de Ferramentas de LLM e Reflexão Crítica

8.1 Resumo do Diário e Interações Relevantes

Ao longo do projeto, foram registradas 5 interações formais com ferramentas de Inteligência Artificial (Claude Opus 4.6 Thinking e Gemini 3.8 Flash), documentadas integralmente em docs/llm.md. O suporte incluiu o desenho preliminar da hierarquia de planos, esclarecimento teórico sobre o mecanismo de despacho polimórfico em PHP, modelagem da composição entre Aluno e Treinos e estruturação limpa do arquivo public/index.php.

8.2 O Erro da LLM Percebido e Corrigido

Na Interação 3, a LLM sugeriu a utilização da função mb_strtolower() para padronizar os dias da semana em src/Treino.php. Ao executar php teste.php, o interpretador PHP abortou imediatamente com erro fatal de função indefinida (*Fatal error: Uncaught Error: Call to undefined function mb_strtolower()*). Identifiquei que a extensão mbstring não vem habilitada por padrão em muitas instalações do PHP para Windows. Corrigi o problema substituindo a função por strtolower(trim($diaSemana)) e padronizando os dias em strings ASCII pura (ex.: 'terca', 'sabado'). Isso garantiu portabilidade nativa sem qualquer dependência externa.

8.3 Onde Ajudou, Onde Atrapalhou e Lições Aprendidas

A LLM foi de grande valia como catalisadora de raciocínio, ajudando a antecipar regras de validação e acelerando a escrita de testes. Por outro lado, mostrou tendência a sugerir dependências desnecessárias e métodos *setter* que enfraqueciam o encapsulamento. A lição central é que a IA potencializa o programador qualificado, mas exige vigilância crítica e domínio conceitual para auditar e corrigir o código gerado.

9. Dificuldades, Aprendizados e Conclusão

9.1 Dificuldades Enfrentadas

O maior desafio prático consistiu em projetar a camada web sem recorrer a frameworks utilitários, garantindo simultaneamente o roteamento limpo via query string, tratamento de erros via mensagens de sessão/variáveis de estado e a implementação da persistência em arquivo JSON com correta desserialização de subclasses polimórficas.

9.2 Principais Aprendizados

O projeto permitiu fixar com clareza o ciclo de vida completo de uma aplicação web: a requisição HTTP trafegando dados via GET ou POST, o isolamento dos objetos de domínio com forte tipagem e a segurança proporcionada por htmlspecialchars() contra vulnerabilidades de injeção.

9.3 Conclusão

O sistema **FitBoa Viagem** foi integralmente construído e homologado em estrita concordância com as diretrizes do edital da disciplina de Programação Web I. Todos os requisitos de orientação a objetos (OO1 a OO8), camada web (W1 a W5) e o requisito bônus de persistência foram superados com sucesso, resultando em uma base de código sólida, elegante e plenamente pronta para avaliação.